

ANDROID STUDIO

KOTLIN-BALDINTZAZKO AGINDUAK

HELBURUAK

Gai honetan kontzeptu hauek landuko ditugu:

1. Baldintzazko Aginduak
2. When agindua
3. If agindua

BALDINTZAZKO AGINDUAK

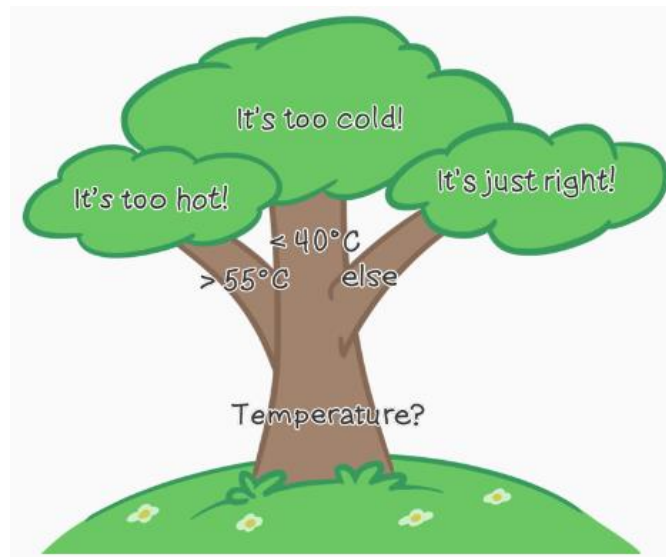
Demagun ondorengo egoera dugula:

Urrezko errizitoak desberdin erreakzionatzen du esnearen temperaturaren arabera:

- ▶ Temperatura 55°C -tik gorakoa denean, honela dio: "Bero handiegia egiten du!"
- ▶ Temperaturak 40°C -tik behera egiten duenean, kexatu egiten da: "Hotza egiten du!"
- ▶ Baina tarteko zerbait egiten baduzu, berak esango du: "Tenperatua perfektua du!".

BALDINTZAZKO AGINDUAK

Kotlin kodea idatz dezakegu esnearen tenperaturaren arabera Urrezko errizitoak zer esango duen adierazteko. Programazioari dagokionez, aurreko hiru kasuei (hau da, taulako hiru errenkada desberdinei) **adar** esaten zaie. Erabilgarria bazaizu, adarrak ikus ditzakezu zuhaitz batean ikusiko zenituzkeen bezala:

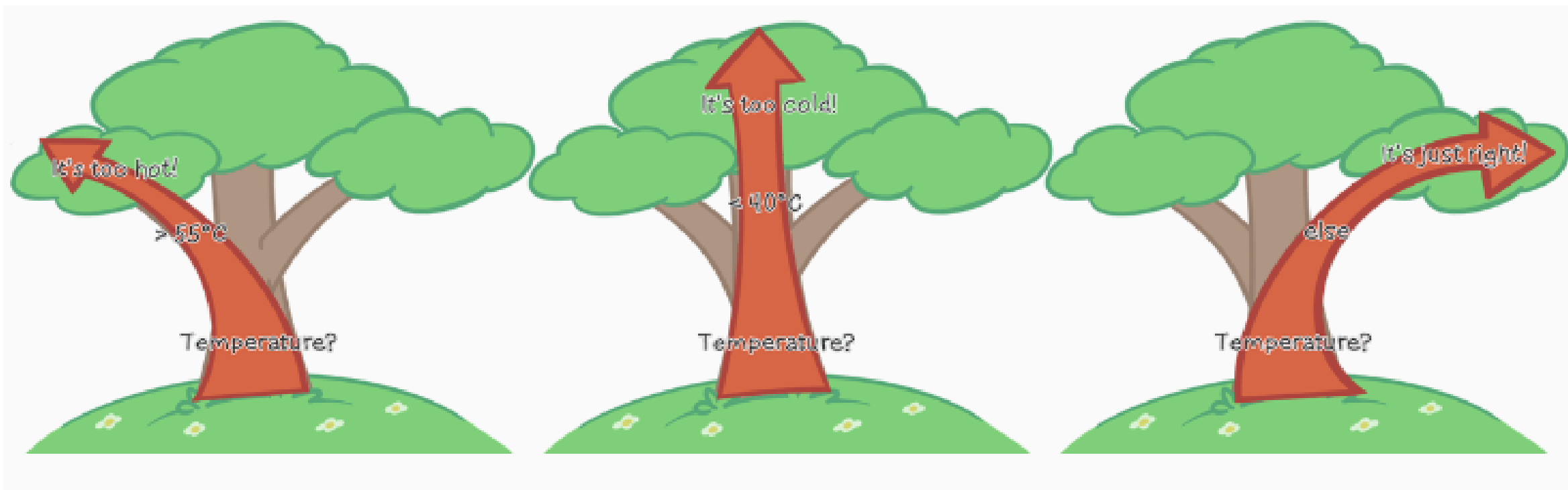


BALDINTZAZKO AGINDUAK

Orain arte, idatzi dugun kodea exekutatu dugunean, kode-lerro bakoitza exekutatu da. Baina orain, kodea idazten hasiko gara, eta adar bakarra exekutatuko da aldi bakoitzean. Kodea exekutatzen den bitartean, exekutatzen den ibilbideari (hartzen duen adar partikularra barne) egikaritze-bidea deitzen zaio.

Hemen ikus ditzakezu gure Kotlin kodeak har ditzakeen hiru exekuzio-ibilbideak, temperaturaren arabera:

BALDINTZAZKO AGINDUAK



BALDINTZAZKO AGINDUAK

Adarren artean baldintzaren baten arabera (adibidez, temperatura) aukeratzeko dituzten kode-erakuntzei **baldintzazkoak** deitzen zaie.

Baldintzako aginduak idazteko Kotlin-ek `if` para bat aukera ditu.

Ondorengo taula erabiliko dugu adibide gisa.

Tenperatura	Erantzuna
> 55	Bero handiegia egiten du!
<40	Hotza egiten du!
Tartekoa	Tenperatura perfektua da!

BALDINTZAZKO AGINDUAK

Honelako taula bat irakurtzeko eta ulertzeko erraza da. Ezkerreko zutabea zeharkatu eta, Urrezko Errizitoak edaten ari den esnearen tenperatura aurkitzen dugunean, eskuinera begiratuko dugu nola erreakzionatuko duen ikusteko. Kotlinek **when** izeneko baldintza ahaltsu bat eskaintzen digu, taula hori gure kodean idazteko aukera ematen diguna. Horrela ikusten da.

```
val temperature = 48

val reaction = when {
    temperature > 55 -> "It's too hot!"
    temperature < 40 -> "It's too cold!"
    else                -> "It's just right!"
}
```

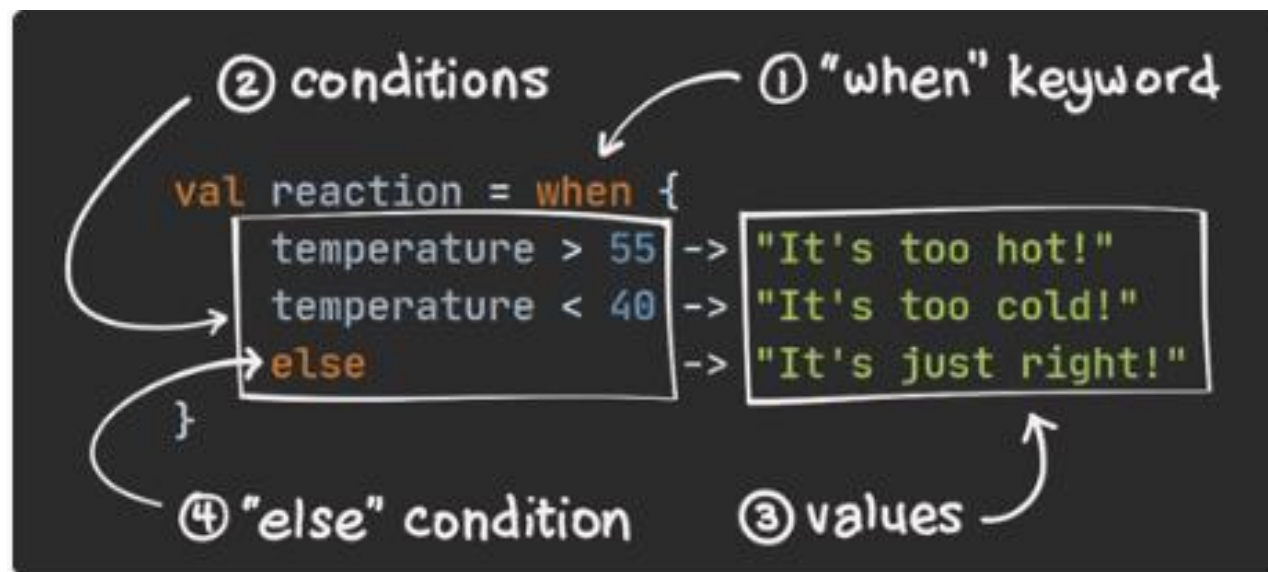
BALDINTZAZKO AGINDUAK

Ikus dezakezunez, aurreko taularen ia berdina dirudi honek. Irekitzeko giltzaren eta ixteko giltzaren arteko lerro bakoitza gure taulako errenkada bat baino ez da. Erantzunari balio desberdina esleituko zaio tenperatura aldagaiaren balioaren arabera:

- ▶ Tenperatura 55 baino handiagoa denean, orduan erantzuna(reaction) "Bero handiegia egiten du!" esleituko da.
- ▶ Tenperatura 40 baino txikiagoa denean, orduan erantzuna (reaction) "Hotza egiten du!" esleituko da.
- ▶ Bestela, erantzuna (reaction) "Tenperatura perfektua da!" esleituko zaio.

BALDINTZAZKO AGINDUAK

Benetan **when** ulertzeko, ikus ditzagun zatiak:



BALDINTZAZKO AGINDUAK

Konparazio eta berdintasun eragileak:

Adibide horietako bakoitzak Boolean balio bat ebaluatzen du, adierazpena egiazkoa den, adierazten duena.

Operador	Descripción	Ejemplo
<	Menos que	<code>x < 5</code>
>	Más que	<code>x > 5</code>
==	Igual a	<code>x == 5</code>
<=	Menor o igual a	<code>x <= 5</code>
>=	Mayor o igual a	<code>x >= 5</code>
!=	No es igual a	<code>x != 5</code>

BALDINTZAZKO AGINDUAK

When hitza nahiko erabilgarria da, eta ziurrenik sarritan erabiliko duzue. Ezaugarri interesgarri batzuk ere baditu. Ikus ditzagun horietako batzuk orain:

► Benetako lehen baldintzak irabazten du.

```
val temperature = -5

val reaction = when {
    temperature > 55 -> "It's too hot!"
    temperature < 40 -> "It's too cold!"
    temperature < 0  -> "It's frigid!"
    else              -> "It's just right!"
}
```

Honen emaitza **It's too cold!** da.

BALDINTZAZKO AGINDUAK

It's frigid! Eraitza lortu nahi izanez gero aginduen ordena aldatu beharko litzateke.

```
val temperature = -5

val reaction = when {
    temperature > 55 -> "It's too hot!"
    temperature < 0  -> "It's frigid!"
    temperature < 40 -> "It's too cold!"
    else              -> "It's just right!"
}
```

BALDINTZAZKO AGINDUAK

Java edo C bezalako beste lengoaia batzuetan kodea idatzi baduzue, baliteke **Switch** moduko adierazpen bat erabili izana.

Switch eta Kotlin-eko **When** adierazpenaren arteko alde garrantzitsu bat da switch bidez aitortzen huts egindako exekuzio-metodo bat dutela, eta horrek esan nahi du baldintza bat datorrenean, haren azpiko adierazpen guztiak (baita beste baldintza batzuetarako aitortzen diren ere) exekutatu egingo direla, **Break** adierazpena erabiltzen ez baduzu behintzat.

Kotlin-en, **when** ez dago trantsiziorik, beraz, kasu bakarra exekutatu da.

BALDINTZAZKO AGINDUAK

► Kointzidentzia zehatzak.

Batzuetan, balio-tarte bat bilatu beharrean (adibidez, "55 baino handiagoa den edozein balio" edo "40 baino txikiagoa den edozein balio"), kointzidentzia zehatz bat bilatzen dugu.

Demagun ondorengo taula dugula:

Eskaera kopurua	Liburuaren Prezioa
1	19,99€ bakoitza
2	18,99€ bakoitza
3	16,99€ bakoitza
4	16,99€ bakoitza
≥ 5	14,99€ bakoitza

BALDINTZAZKO AGINDUAK

When enuntziatua honela idatz genezake:

```
val quantity = 3

val pricePerBook = when {
    quantity == 1 -> 19.99
    quantity == 2 -> 18.99
    quantity == 3 -> 16.99
    quantity == 4 -> 16.99
    else           -> 14.99
}
```

BALDINTZAZKO AGINDUAK

Hala ere, Kotlinek hau are gehiago laburtzeko aukera emango digu, gai bat zehazteko aukera emanez. Horrela ikusten da:

```
val quantity = 3

val pricePerBook = when (quantity) {
    1    -> 19.99
    2    -> 18.99
    3    -> 16.99
    4    -> 16.99
    else -> 14.99
}
```

BALDINTZAZKO AGINDUAK

Gehiago ere labur daiteke:

```
val quantity = 3

val pricePerBook = when (quantity) {
    1    -> 19.99
    2    -> 18.99
    3, 4 -> 16.99
    else -> 14.99
}
```

BALDINTZAZKO AGINDUAK

► Baldintza bakoitza kontuan hartu behar da

When, Kotlinen adierazpen honek balio posible guztietarako baldintzak izan behar ditu. Horregatik esaten dugu **when zehatza** izan behar dela. Horregatik, normalean, **else** baldintza sartu behar da amaieran. Batzuetan, zehatza izan daiteke, **else** erabili gabe. Adibidez, kontuan hartu adierazpen boolearra. badakigunez, balio boolearrek bi balio posible baino ez dituzte: *true* eta *false*. Kotlinek badaki balio boolearra *true* edo *false* baino ezin dela izan. Beraz, bi kasuetarako adar bat dugun bitartean, ez dugu **else** bat beharko.

```
val isLightbulbOn = true

val reaction = when (isLightbulbOn) {
    true -> "It's bright"
    false -> "I can't see"
}
```

BALDINTZAZKO AGINDUAK

Gehiago ere labur daiteke:

```
val quantity = 3

val pricePerBook = when (quantity) {
    1    -> 19.99
    2    -> 18.99
    3, 4 -> 16.99
    else -> 14.99
}
```

BALDINTZAZKO AGINDUAK

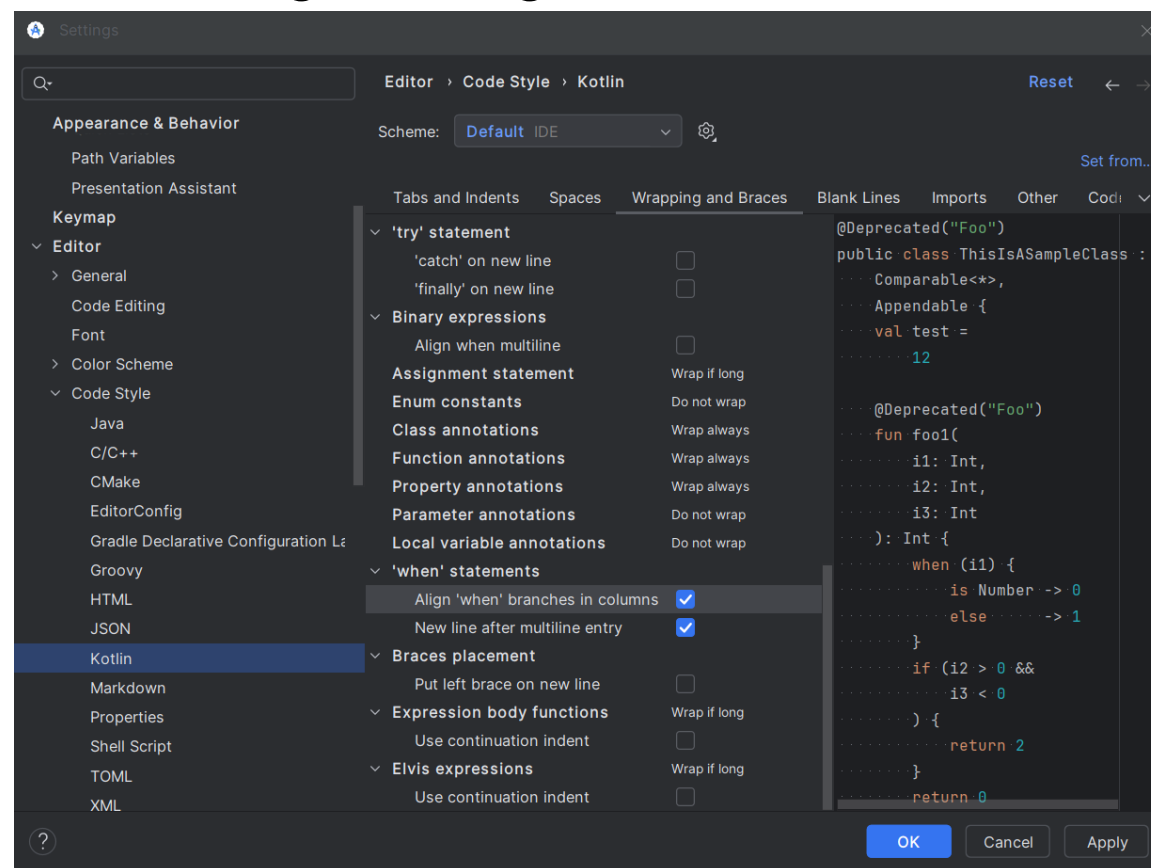
Aholku azkarra

```
val reaction = when {  
    temperature > 55 -> "It's too hot!"  
    temperature < 0 -> "It's frigid!"  
    temperature < 40 -> "It's too cold!"  
    else -> "It's just right!"  
}
```

```
val reaction = when {  
    temperature > 55 -> "It's too hot!"  
    temperature < 0 -> "It's frigid!"  
    temperature < 40 -> "It's too cold!"  
    else -> "It's just right!"  
}
```

BALDINTZAZKO AGINDUAK

Hau lortu ahal izateko ondorengo ezaugarriak aldatu behar dira:



BALDINTZAZKO AGINDUAK

When adierazpena ahaltsua da eta, Kotlinen garatzaile gisa, asko erabiliko duzue. Hala ere, gehiegizkoa izan daiteke egiaztapen boolear sinpleetarako. Kasu horietarako, Kotlinek bigarren baldintza-mota bat eskaintzen digu

IF

BALDINTZAZKO AGINDUAK

Bonbillaren gure adibidea are zehatzagoa izan daiteke **IF** adierazpena erabiliz **When** ordeztuz. IF adierazpenak when adierazpenak bezalakoak dira, baina baldintza bakarra eta bi kasu dituzte: bat truerako eta bestea falserako. Horrela ikusten da:

```
val isLightbulbOn = true

val reaction = when (isLightbulbOn) {
    true  -> "It's bright"
    false -> "I can't see"
}
```

```
val reaction = if (isLightbulbOn) "It's bright" else "I can't see"
```

BALDINTZAZKO AGINDUAK

Eta hona hemen atalak:

```
val reaction = if (isLightbulbOn) "It's bright" else "I can't see"
```

Diagram illustrating the components of the `if` statement:

- `if` keyword (indicated by a curved arrow)
- `isLightbulbOn` (indicated by an upward arrow, labeled `condition`)
- `"It's bright"` (indicated by an upward arrow, labeled `true branch`)
- `"I can't see"` (indicated by an upward arrow, labeled `false branch`)

Beste era batzuetara idatzia:

```
val reaction =  
    if (isLightbulbOn)  
        "It's bright"  
    else  
        "I can't see"
```

```
var isLightbulbOn = true  
  
val reaction =  
    if (isLightbulbOn) {  
        isLightbulbOn = false  
        "I just turned the light off."  
    } else {  
        isLightbulbOn = true  
        "I just turned the light on."  
    }
```

BALDINTZAZKO AGINDUAK

IF ere erabili daiteko baldintza bat baino gehiago adierazteko:

```
val pricePerBook =  
    if (quantity == 1)  
        19.99  
    else if (quantity == 2)  
        18.99  
    else if (quantity == 3)  
        16.99  
    else if (quantity == 4)  
        16.99  
    else  
        14.99
```

BALDINTZAZKO AGINDUAK

Praktika onak:

- `a = a + 1` se puede escribir como `a += 1`
- `b = b - 2` se puede escribir como `b -= 2`
- `c = c * 3` se puede escribir como `c *= 3`
- `d = d / 4` se puede escribir como `d /= 4`